Learning the Pokémon Showdown Battle System

A self-study guide to the simulator, battle mechanics, and the fastest way to navigate them. Scope: `sim/`, mechanics data, formats/rules, and simulator tests—no UI, chat, matchmaking, ladders, or multiplayer session code.

Snapshot used: `smogon/pokemon-showdown` commit `bb179fbf` (2026-08-27), matching your local `battle_server/core` checkout. Paths below are relative to that directory. Current launch/build scripts enforce Node.js 22 or newer.

The one-sentence mental model

A format selects a generation and rules; the `Dex` assembles the correct data; a `Battle` owns mutable state; each `Side` turns text choices into actions; `BattleQueue` orders them; `BattleActions` executes them; and event handlers on moves, abilities, items, conditions, and rules modify the result.

```
Format → Modded Dex → Battle state → Side choice → Action queue → BattleActions → Events/effects →
                                        State + battle log
```

Your working map

Path	Question it answers
`sim/battle.ts`	Who owns the battle, requests choices, runs the turn loop, dispatches actions, processes events, damage/healing/fainting, and emits the log?
`sim/side.ts`	How is one player's team represented, and how are `move`, `switch`, `team`, and `pass` choices parsed and validated?
`sim/battle-queue.ts`	How do incomplete choices become full actions, and how are switch order, move priority, speed, and ties resolved?
`sim/battle-actions.ts`	What actually happens when a Pokémon switches or uses a move—including targeting, accuracy, immunity, damage, secondaries, and recoil?
`sim/pokemon.ts`, `field.ts`, `dex*.ts`	Where do Pokémon/field state live, and how is the correct generation's data exposed?
`data/*.ts`, `data/mods/*`	Where is each mechanic declared, and how does it differ by generation or custom mod?
`config/formats.ts`, `data/rulesets.ts`	Which mod and battle rules does a named format activate?
`test/sim/**`, `test/common.js`	What behavior is promised, and how can I reproduce a mechanic in a tiny deterministic battle?

Set up a tight feedback loop

```
cd battle_server/core
node --version                # use 22+
npm ci                        # only when dependencies are missing/stale
node build                    # compiles TypeScript into dist/
npx mocha -g "Thunder Wave"   # focused mechanics test
npm test                      # lint + normal tests + TypeScript check
```

Start with `test/sim/moves/thunderwave.js`. Find `thunderwave` in `data/moves.ts`, then follow the generic pipeline only as far as that test requires. One behavior per trace is faster than reading the giant core files top to bottom.

The Runtime Path You Should Memorize

Most battle questions become manageable once you can place them on this path. Read the named functions in this order; ignore adjacent helpers until a concrete behavior leads you there.

1. Construction selects the mechanics universe

`new Battle(options)` resolves the format, calls `Dex.forFormat`, derives the generation/rules, and creates `Field`, side slots, seeded `PRNG`, `BattleQueue`, and `BattleActions`. It mixes in mod scripts, so the same public methods can run different generation algorithms.

2. Players become sides; the battle starts when all required sides exist

`Battle.setPlayer` creates a `Side` and its `Pokemon`. When all slots are filled, `Battle.start` establishes rules, preview, and switch-in setup before the first move request. Supplied teams are assumed valid; legality belongs to `sim/team-validator.ts`.

3. Requests define what a side may choose

`Battle.makeRequest('teampreview' | 'move' | 'switch')` clears choices and creates each `activeRequest` through `getRequests`. Request state is authoritative: moves are rejected during forced switching, and choices are rejected between requests.

4. A choice becomes one or more actions

`Battle.choose('p1', input)` delegates to `Side.choose`, which parses `move thunderwave`, `switch 3`, and comma-separated doubles choices. Specialized methods reject illegal/incomplete input and append partial actions. When all sides finish, `Battle.commitChoices` feeds them to the queue.

5. The queue resolves and sorts

`resolveAction` fills in moves, targets, action order, priority, fractional priority, and speed; one choice may expand (Terastallization plus a move). Sorting uses lower order bucket, then higher priority, higher speed, and seeded tie-breaking. Mid-turn switches are inserted without rebuilding the remaining turn.

6. The turn loop dispatches one action at a time

```
Battle.commitChoices()
  → Side.commitChoices()
  → BattleQueue.addChoice() / sort()
  → Battle.turnLoop()
  → Battle.runAction(action)
  → BattleActions.runMove(...)
  → BattleActions.useMove(...)
  → tryMoveHit / trySpreadMoveHit
  → ordered hit steps → spreadMoveHit → getDamage
```

`turnLoop` shifts actions until empty, ended, or paused for a mid-turn request (U-turn or faint replacement). `runAction` dispatches action types; `endTurn` cleans up, increments the turn, and makes the next request.

7. Events let every active effect intervene

`Battle.runEvent` finds matching handlers across targets/sources, allies/foes, statuses, volatiles, abilities, items, conditions, formats, and rules; orders them by event priority/speed; then calls them. `singleEvent` invokes one named effect directly.

Handler return	Meaning in the common relay pattern
`undefined`	Leave the relay value unchanged and continue.
a number/object	Replace the relay value—for example, alter base power, a stat, or damage.
`false`	Stop/fail, normally allowing the generic failure message.
`null`	Stop/fail silently, commonly because the handler emitted its own message.

Reading rule: for `runEvent('X')`, search `onX`, `onSourceX`, `onAllyX`, and `onFoeX`. For `singleEvent('X', effect)`, inspect that effect's `onX`.

Navigate by Behavior, Not by File Size

The engine is deliberately data-driven. A mechanic may be a plain data field, an event callback, a reusable condition, an engine algorithm, or a generation override. Locate its category before changing or deeply reading anything.

The five places mechanics live

Kind	Examples	Start here
Declarative effect	Power, type, accuracy, status, boosts, target, flags	`data/moves.ts`, `abilities.ts`, `items.ts`
Effect callback	`onHit`, `onDamagingHit`, `onResidual`, `onSwitchIn`	The effect entry in `data/` or `data/mods/`
Shared condition	paralysis, weather, terrain, volatile or side conditions	`data/conditions.ts` or a move's nested `condition`
Core algorithm	choice parsing, action order, accuracy, damage formula, switching, fainting	`sim/side.ts`, `battle-queue.ts`, `battle-actions.ts`, `battle.ts`
Ruleset/mod override	generation math, clauses, metagame-specific behavior	`config/formats.ts`, `data/rulesets.ts`, `data/mods/<mod>/`

Three traces that teach the architecture

Simple data **Thunder Wave.** Its test is `test/sim/moves/thunderwave.js`. The base move declares accuracy, Electric type, `status: 'par'`, and `ignoreImmunity: false`; generic hit/status machinery does the rest. There is no "ThunderWave class."

Condition **Stealth Rock.** The move declares a `sideCondition` with `onSideStart`/`onSwitchIn`. The switch hook checks Boots, calculates Rock effectiveness, and deals fractional max-HP damage—persistent state implemented through events.

Ability hook **Aftermath.** Its `onDamagingHit` checks fainting/contact and calls `this.damage`. `BattleActions` raises the event; the ability subscribes. The engine has no Aftermath branch.

How generations and formats change the answer

`config/formats.ts` maps a format to a `mod`/`ruleset`. `Dex.forFormat` follows `Scripts.inherit` and merges entries marked `inherit: true`. Mods can patch data/callbacks or replace whole battle, Pokémon, or action methods. Gen 1, for example, inherits Gen 2 while overriding both scripts and individual moves. Always check the active mod.

A repeatable search recipe

```
# 1. Find tests and data entries (IDs are lowercase and punctuation-free)
rg -n "Thunder Wave|thunderwave" test/sim data config/formats.ts

# 2. Find callbacks and the engine event that raises them
rg -n "onDamagingHit|runEvent\('DamagingHit'" data sim

# 3. Check generation overrides
rg -n "thunderwave:|runMove\(" data/mods

# 4. Run only the behavior you are tracing
npx mocha -g "Thunder Wave"
```

Route common questions quickly

If you are asking…	Inspect…
Why did this move fail or miss?	`BattleActions.runMove/useMove`, hit-step methods, then relevant `Try*`, immunity, and accuracy handlers.
Why this damage number?	`BattleActions.getDamage` plus `BasePower`, offensive/defensive stat, effectiveness, critical, and `ModifyDamage` events.
Why did this act first?	`BattleQueue.resolveAction/sort`, `Battle.getActionSpeed`, and priority/speed event handlers.
Where is this persistent state?	`Pokemon.volatiles/status`, `Side.sideConditions`, or `Field.weather/terrain/pseudoWeather`.

A Practical Study Plan and Debugging Method

Do these labs in order. Each should end with a sentence you can prove from code and a focused test you can run. That turns familiarity into a durable model.

Lab 1 — Declarative move (30–45 min)

1. Read `test/sim/moves/thunderwave.js`.
2. Inspect `thunderwave` in `data/moves.ts`.
3. Follow type immunity, accuracy, and status application in `battle-actions.ts`/`pokemon.ts`.
4. Run `npx mocha -g "Thunder Wave"`.

Goal: explain why a Ground target stays un-paralyzed without finding any Thunder Wave-specific engine branch.

Lab 2 — Event ability (45–60 min)

1. Read `test/sim/abilities/aftermath.js`.
2. Read the ability's `onDamagingHit`.
3. Find where `DamagingHit` is raised and how contact flags are tested.
4. Change only the test teams in a scratch branch: compare a contact move, non-contact move, and a protected source.

Goal: distinguish an event publisher from an effect subscriber.

Lab 3 — Persistent condition (45–60 min)

1. Read the Stealth Rock move and nested condition.
2. Find how `Side.addSideCondition` stores its state.
3. Trace the next switch through queue → switch action → `SwitchIn`.
4. Run its tests with `npx mocha -g "Stealth Rock"`.

Goal: identify the owner, lifetime, start hook, trigger hook, and removal path of a condition.

Lab 4 — Turn order (60 min)

1. Use `test/sim/misc/turn-order.js` as the entry point.
2. Trace `Side.commitChoices` → `BattleQueue.resolveAction` → `sort`.
3. Compare a switch, normal-priority move, priority move, and speed tie.
4. Record the fixed seed; repeat the test to prove determinism.

Goal: predict queue order before running the test.

Lab 5 — Cross-generation behavior (60–90 min)

1. Choose one move changed in Gen 1 (Bite is a compact example).
2. Compare base `data/moves.ts` with `data/mods/gen1/moves.ts`.
3. Use `common.gen(1)` and the base test tools to run the same scenario twice.
4. Explain the `inherit: true` merge and which Dex each battle owns.

Goal: stop treating the latest-generation data as universal.

Lab 6 — Your own regression (60 min)

1. Pick one interaction you care about.
2. Copy the nearest small test; use minimal teams and deterministic moves.
3. Assert state, not prose: HP, status, boosts, item, queue, or log line.
4. Use `battle.getDebugLog()` only while diagnosing; call `battle.destroy()` in cleanup.

Debugging discipline

- **Start from an observable.** State the exact wrong HP/status/order/log output, then reduce to the smallest battle.
- **Control randomness.** Use the deterministic test seed, avoid speed ties, or explicitly use `forceRandomChance`. Never add `Math.random()` to mechanics.
- **Respect event pathways.** Direct state mutation can bypass immunity, suppression, messages, and downstream hooks. Prefer the established battle/Pokémon methods.
- **Check the mod.** Reproduce in the same format/generation before editing base logic.
- **Test narrow, then broad.** Run `npx mocha -g "exact title"`, then the relevant test file/group, then `npm test`; use `npm run full-test` before a substantial upstream contribution.

You understand the engine when you can…

- ☐ trace a choice from `Side.choose` to its final state/log changes;
- ☐ name which object owns a status, volatile, side condition, weather, or queue action;
- ☐ find both an event handler and the engine site that raises its event;
- ☐ explain how a format selects a mod and how inherited data changes a mechanic;
- ☐ write a deterministic regression test that proves one interaction.

Primary references: sim/SIMULATOR.md, sim/DEX.md, test/TESTS.md, and the source files named throughout this guide.